Assignment-2: Path Tracer

Milan Lagae

University Name: Vrije Universiteit Brussel

Faculty: Sciences and Bioengineering Sciences

Course: Performance Analysis & Evaluation

May 27, 2026

Contents

1. Intro

This report will discuss the improvements made to the CPU based pathtracer for Assignment-2 of the course: 'Performance Analysis & Evaluation'. Each major improvement will be discussed in their respective sections: Lock in Section 3; Inlining in Section 4; Pool in Section 5; Parallel in Section 6. The results in each section will be the improvement added during that section onto the previous version.

Finally, the latest version of the implementation will be thoroughly discussed in Section 8. The methodology Section 2 discusses deviations from the standard. The CoV charts can be found in Section 10.2.

2. Methodology

This section will briefly explain some of the benchmarking methodology used and why there are some deviations from the recommendations made in class. During the benchmarking, information was collected using `taskset` & `perfstat`. The list of PMU events tracked during benchmarking are the following: `branches`, `branch-misses`, `cache-misses` `cache-references`, `cycles`, `instructions`, `stalled-cycles-frontend`.

Due to `benchkit` limitation (or not finding how to), the `cpu_list` variable was set to all cores available for each iteration. The ideal would be that the `cpu_list` matches the `nb_threads` variable, but this was unsuccessful in implementation.

The confidence intervals & CoV charts can be found in Section 10.2. The only noticeable remark is the CoV value(s) for the smallest scene 01 are outside of the recommended ranges. For the other scenes, the values are within acceptable range. The CoV values for the `perf stat` measurements are within the $[70.4, 71.9]$ range.

2.1. Base

The numbers of runs was set to **3**, this deviates from the recommendations [1], [2], but due to how much time the implementation takes, this was considered an appropriate middle ground. Additionally, for `scene-05`, the number of thread count was limited to **20**, for the same reason as before.

2.2. Lock

For the lock improvement, the usage of `taskset` was identical as with the base version. The number of runs was capped at **1** for each iteration. It is acknowledged that the number of runs is considered inappropriate for proper comparison [3]. For `scene-05` the decision was made to only benchmark for the thread range of: 4 until 20.

2.3. Inline

The number of runs per iteration was again set to **1**. It is acknowledged that the number of runs is considered inappropriate for proper comparison. All scenes were executed with the whole thread count range: `[1..33]`.

2.4. Pool

Due to the improvement of the execution time of the application, the decision was made here to set the number of runs for each iteration to **5**, more closely matching the recommendation seen in class. All scenes were executed with the whole thread count range: `[1..33]`.

2.5. Parallel

Due to the limitation of benchkit mentioned earlier, or not finding a working implementation, the build phase of the pathtracer uses the maximum available of threads it can see on the system.

If `taskset` could be set in step with the number of threads given to the render time, the build time would also gradually decrease when the thread count is increased. In the current implementation & benchmarking, the build time uses the maximum number of available threads on the system.

2.6. SoA

For this improvement the decision was made to execute **10** runs per iteration.

2.7. Final

For this improvement **15** runs were executed per iteration.

3. Improvement 1: Lock Contention

This section will discuss the improvement of removing lock contention in the pathtracer implementation.

3.1. Problem

The first problem that was identified & solved is the lock contention in the `random.c` file. When first benchmarking the application, it became clear that increasing the thread count made the application significantly slower.

Initial sourcing for this became clear after using `perf stat`, `perf record` & `strace` for which the result can be seen in Figure 2. For scene 04, with 20 threads: (04-20).

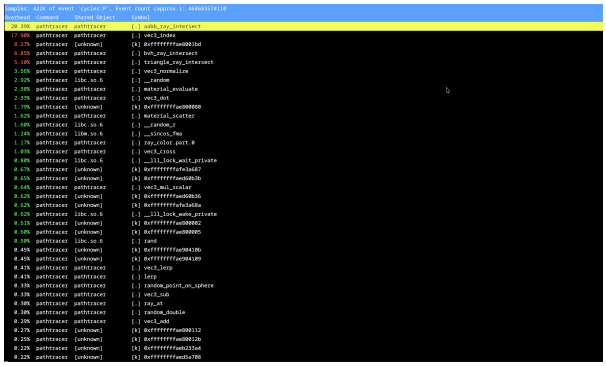

Figure 1: Perf Record - Overhead (04-20)

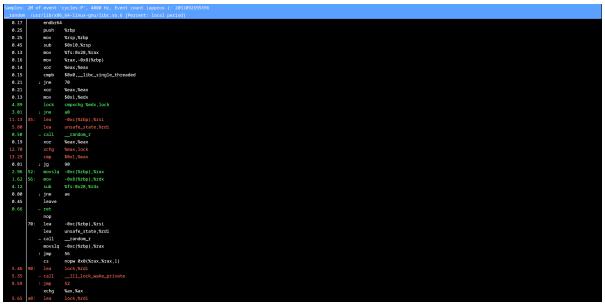

Figure 2: Perf Annotate - Lock (04-20)

This same behavior is visible in Figure 4 where the gray lines between the thread activity is the thread waiting for a lock to be finished.

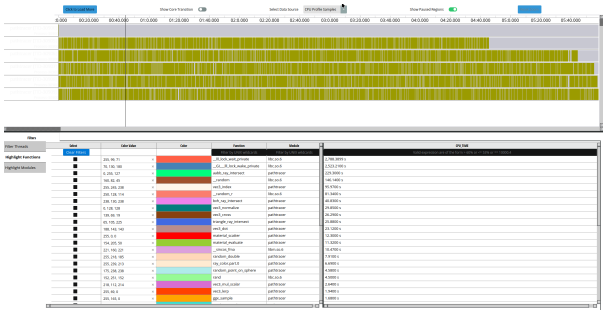

Figure 3: AMD µProf - Lock Time (04-20)

Figure 4: AMD µProf - Lock Contention Threads (04-20)

Using the `perf record` annotate, the offending function can be found: `rand()`, as shown in Figure 2. Looking up the man page of the function [4], it becomes clear that the function is not thread safe and suffers from heavy lock contention in thread heavy workloads.

This behavior becomes clear when the Total Time of each scene is plotted vs the thread count in Figure 5.

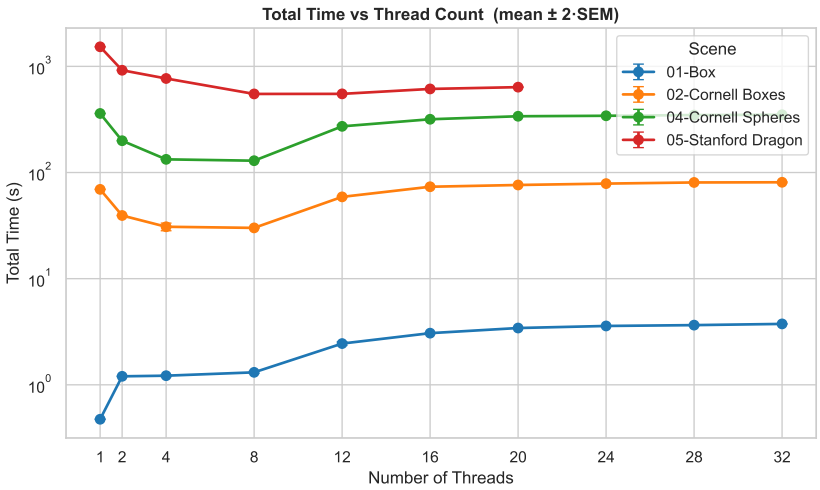

Figure 5: Total Time - Base

In the scenes 01, 02 and 04 increasing the thread count from 8 to 12, increases the total time taking for rendering said image. For the larger (scene 05), this behavior is not as clearly visible, but there is a slight increase in time when increasing the number of threads. This increase can be attributed to the use of the non reentrant safe `rand()` function.

3.2. Solution

Included in the man page of the `rand()` function is a recommendation to use the safe function `rand_r()`, but this function seems to require enabling some posix standard and is deprecated.

As a fix, the `rand()` was replaced with a `xoroshiro128plusplus` implementation based on [5] and `splitmix` for seeding the random number generator [6].

3.3. Results

The improvements in performance when the lock contention is removed, is visible in Figure 6 and Figure 7.

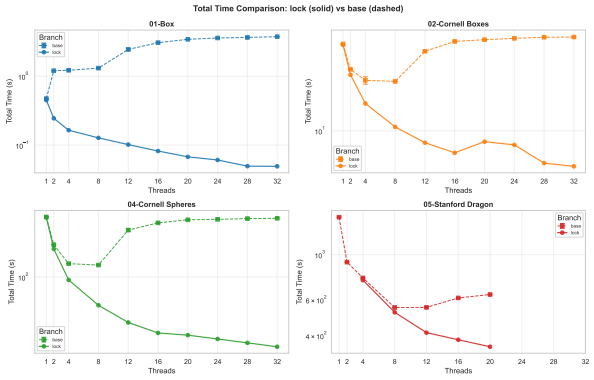

Figure 6: Total Time - Lock vs Base

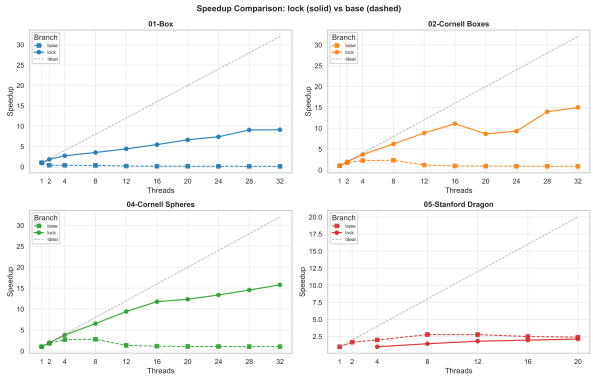

Figure 7: Speedup Total Time - Lock vs Base

The behavior where the speedup increases when more threads are used, clearly shows the non re-entrant behavior of the original `rand()` function. The charts in Figure 7 give an indication of how efficiently the implementation makes use of the available threads compared between the versions.

The improvement in performance becomes even more clear, when looking at the IPC of the two implementations in Figure 8.

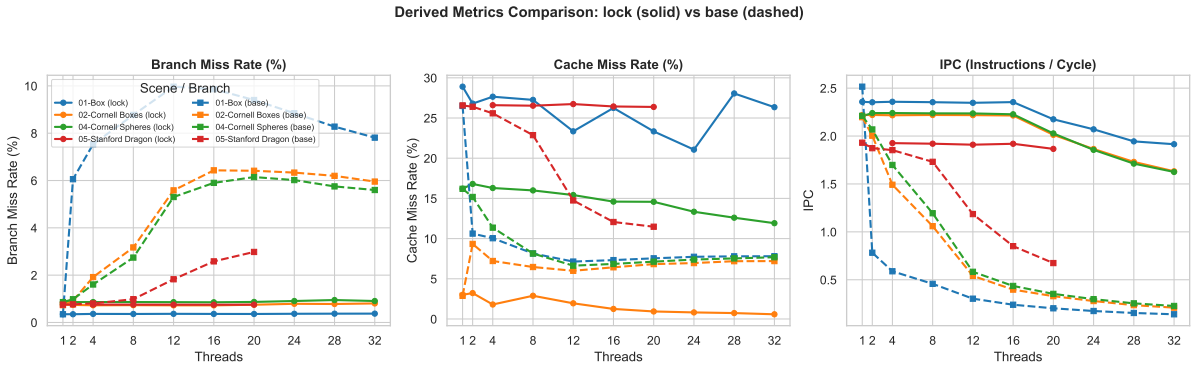

Figure 8: IPC, Branch, Cache Miss Rate Percentage - Lock & Base

4. Improvement 2: Inlining

This section will discuss the process of inlining [7] compute & time intensive `vec3_*` operations.

4.1. Problem

What became clear during the previous improvement & looking at the code, is the heavy usage of `vec3` operation calls. Using the information displayed in Figure 9 & Figure 10, it becomes clear that there is potential performance to be gained.

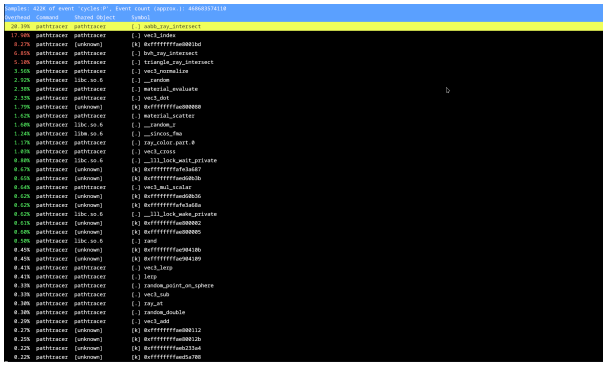

Figure 9: Perf Record - Overhead Vec3 (02-20)

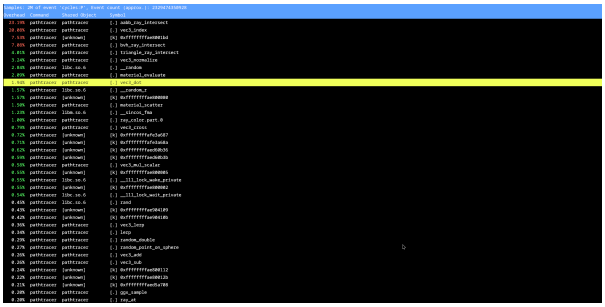

Figure 10: Perf Record - Overhead Vec3

When using the AMD µProf profiling application of which the results are shown in Figure 11, is that the `vec3_index` function alone takes **95 seconds**.

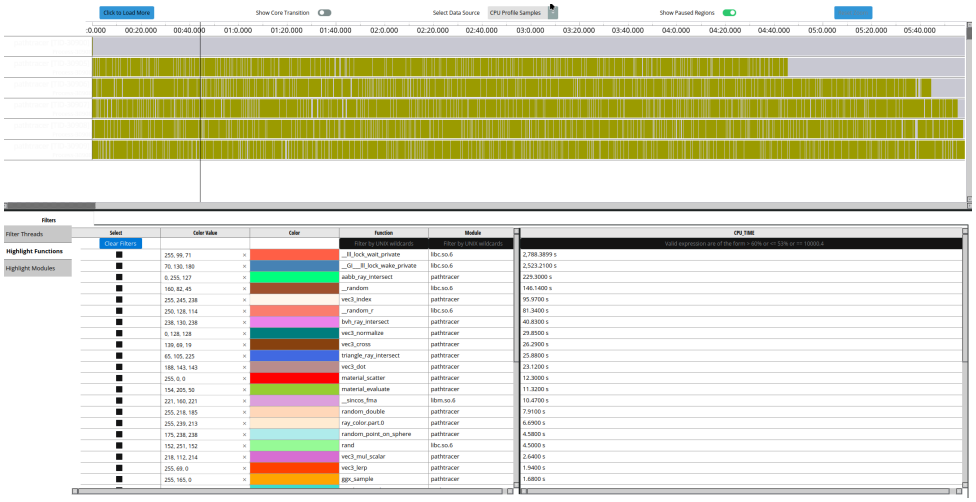

Figure 11: AMD µProf - Vec3 Operations Time (04-20)

4.2. Solution

The most immediate fix for this performance issue is the application of placing all the `vec3_*` functions inside of the `vec3.h` file and placing the `inline` keyword before each function. This 'forces' the compiler to inline the function code in every location where the code is called, – preventing the compiler from having to use `jumps/goto` – and performing (registers updates, etc) for each `vec3` call.

4.3. Results

Both, the time and speedup are charted in Figure 12, Figure 13 respectively. Note that for the speedup in Figure 13, the 'speedup' is compared against the time available of the lowest thread count value.

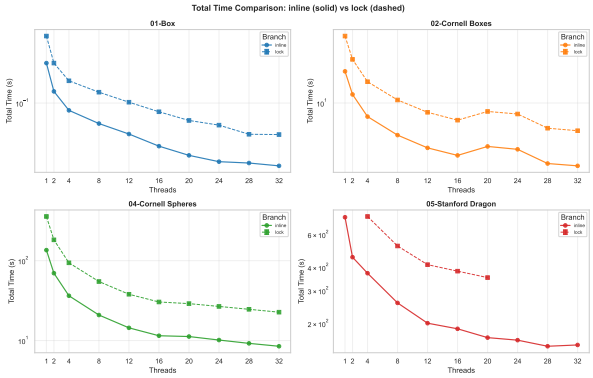

Figure 12: Total Time - Inline vs Lock

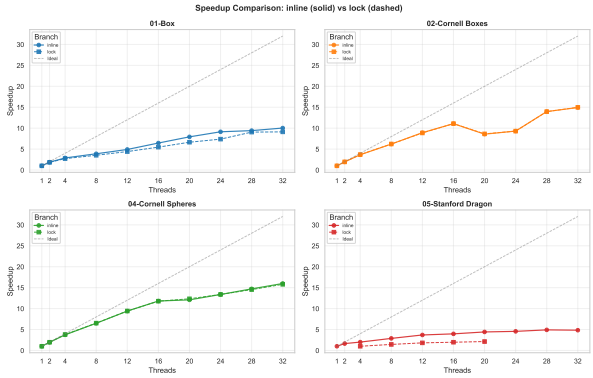

Figure 13: Speedup Total Time - Inline vs Lock

Across the board, there is a general reduction in the time needed for rendering all scenes. The increase in time, when increasing the thread count for scene 02, is visible in both states.

The behavior of the application, regrading speedup (= efficiency of thread usage) does not drastically change between the previous version and the current version. All of the smaller scenes almost completely overlap their respective plots. The largest scene 05, deviates from this, by slightly being more efficient in using more threads.

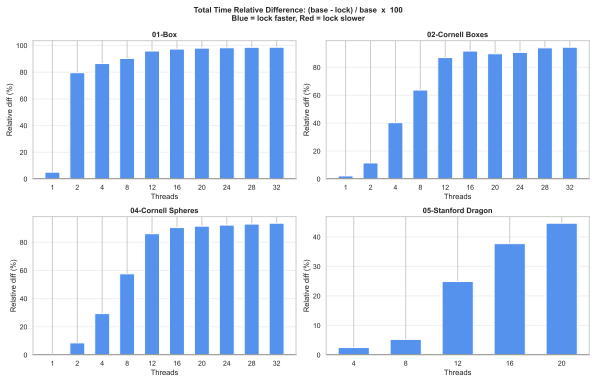

Figure 14: Total Time Relative Difference - Lock vs Base

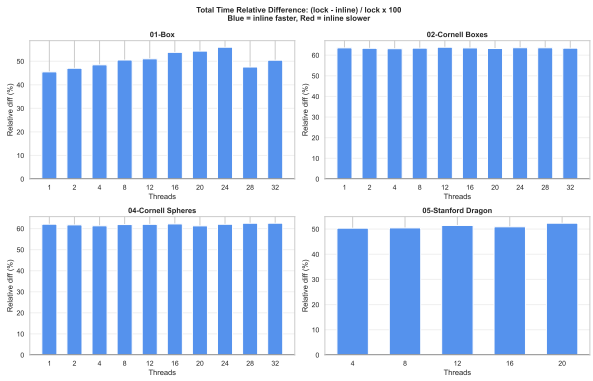

Figure 15: Total Time Relative Difference - Inline vs Lock

These results do not particularly surprise, since the change does not concern itself with more efficient thread usage. The difference in time reduction between thread count is clear when looking at the relative differences in Figure 14 & Figure 15.

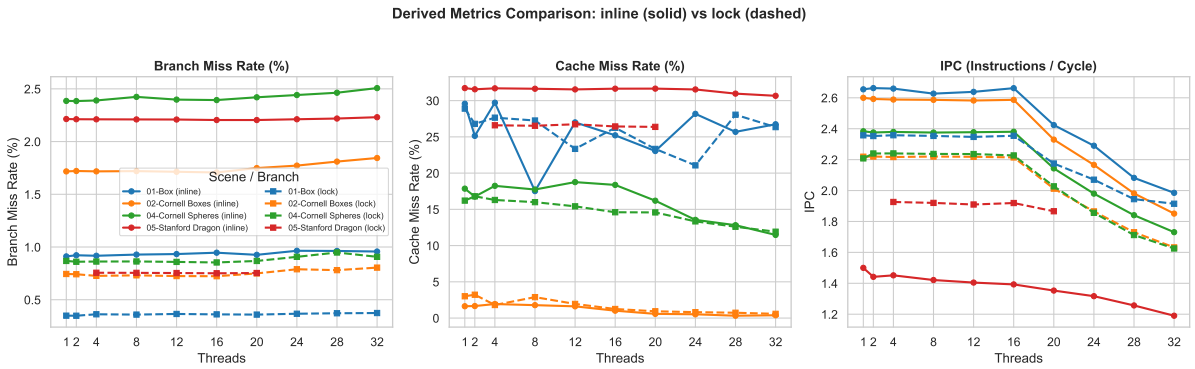

Figure 16: IPC, Branch, Cache Miss Rate Percentage - Inline vs Lock

For all scenes, there is a *'large'* increase in the branch miss prediction rate, while the cache miss rate did not substantially change between the previous version. For the IPC, there is a substantial reduction to note for the largest scene 05.

5. Improvement 3: Thread Pooling

This section will discuss the implementation of adding a thread pool which contains image rendering tasks. Each task will be responsible for a square tile of the image to be rendered.

5.1. Problem

Analyzing the application, using the AMD μProf [8] application, and viewing the thread section, shows the behavior visible in Figure 18. Specifically the end of the timeline of the renderer is important. The illustration in Figure 17 shows how the renderer splits the image in non equal workload, by splitting the image vertical slices from top to bottom.

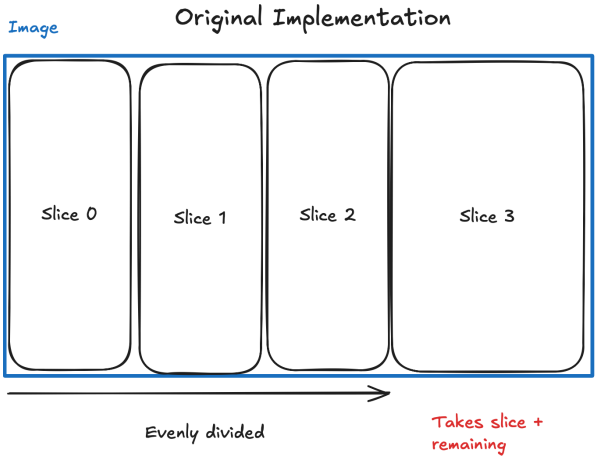

Figure 17: Renderer - Image Slicing Before

Figure 18: AMD μProf - Thread behavior

Depending on the image to be rendered, some threads may have less work than other threads. This because not all pixels in an image are identically or have equal amount of work to be rendered. Additionally, the last thread of the image is padded so it rendered the full width of the image.

5.2. Solution

The problem identified above was solved by two additions. First, instead of vertical slices, the image is now split in smaller (16x16) square tiles, as illustrated in Figure 19. In addition to the image tilling, render tasks are now based on pool design [9], [10].

The improvement in evenly shared work between the threads can be seen in Figure 20. At the end of the timeline, the threads converge to finish simultaneously, eliminating any idle time.

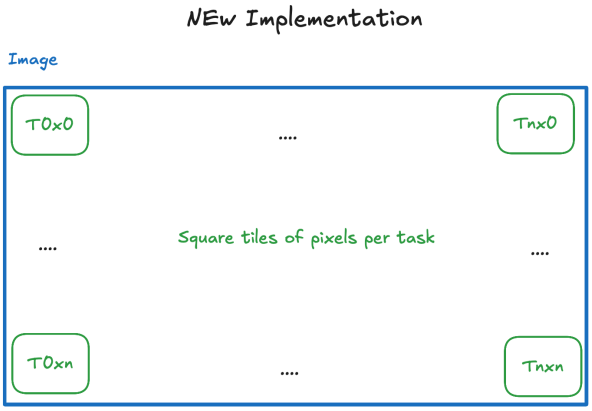

Figure 19: Image Slicing - Vertical Slices

Figure 20: AMD μProf - Thread(s) Non Idling Timeline (04-20)

Included in the pooling modifications is the addition of the `ray_color2_soa` and packing the results of several ray's in an array. The framebuffer of the image has also been transformed into a SoA layout.

5.3. Results

There is *small* reduction in execution time for all scenes with the added improvement, as shown in Figure 21. For scene 02, the 'bulge' that is visible for the treads 16,28 has been eliminated. On glance, the greatest impact of the pool on the execution time is for the largest scene, scene 05.

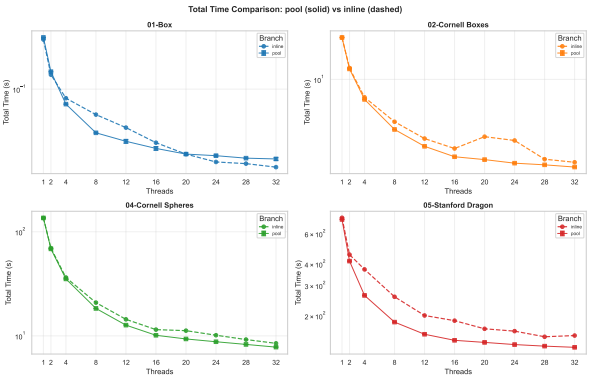

Figure 21: Total Time - Pool vs Inline

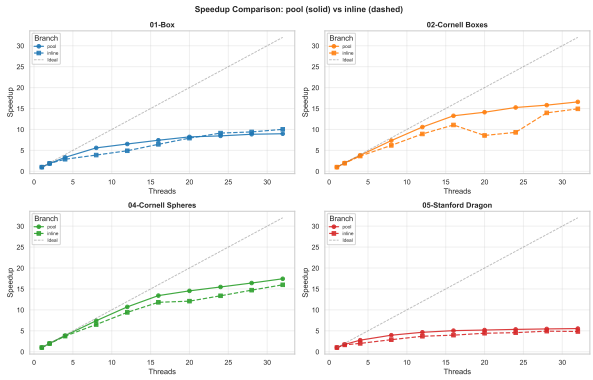

Figure 22: Speedup Total Time - Pool vs Inline

Looking at the speedup in Figure 22, the efficiency of using more threads has been increased, with an overall much better usage for scene 02, where the reduction for the 20,24, threads has been eliminated.

For the IPC, the lowest bound on scene-05 for 32 threads, compared to the previous improvement has been reduced from ∼1.2 to ∼1.1.

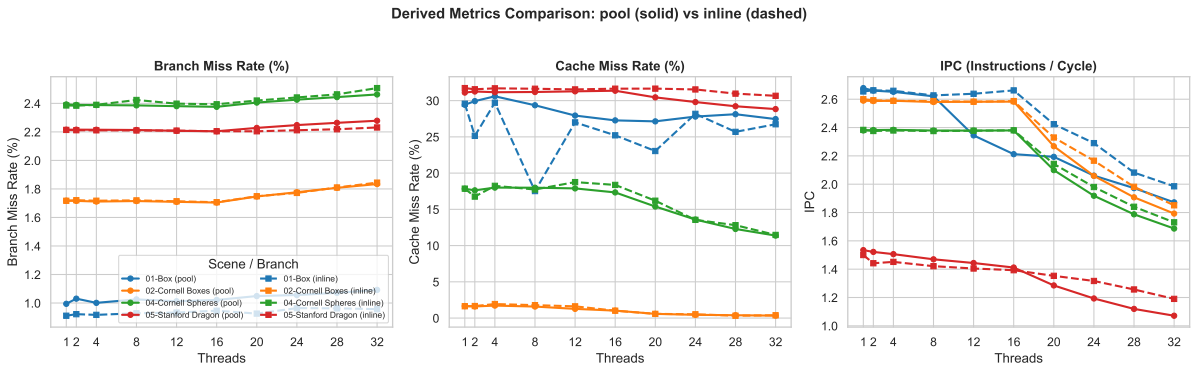

Figure 23: IPC, Branch, Cache Miss Rate Percentage - Pool vs Inline

6. Improvement 4: Parallel Build

This section will focus on making the `bvh_build` initialization function faster, by applying fork-join like multi-threading to the build process.

6.1. Problem

Increasing the complexity & number of triangles when the scenes increases, leads to an explosion of the time it takes to build the BVH tree during the initialization process.

6.2. Solution

This solution consists, of using a separate thread to compute the right side of the current node, while the current thread computes the left side. There is a spawn parallel condition, which guards between the thread overhead creation and compute remaining.

6.3. Results

For the parallel improvement, the total time reduction for the smaller scenes is not really noticeable. With time plots for the scenes 01,02 and 04 matching their previous version. The largest reduction in execution time is for scene 05, as can be seen in Figure 24.

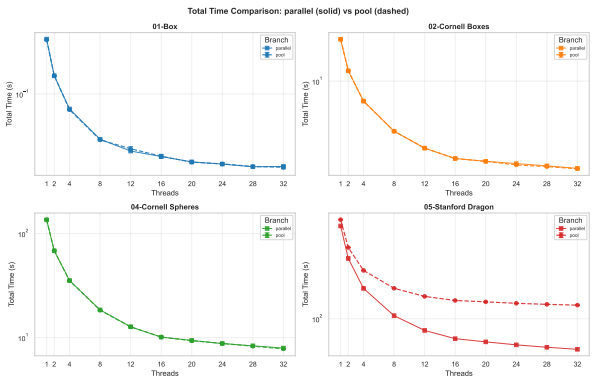

Figure 24: Total Time - Parallel vs Pool

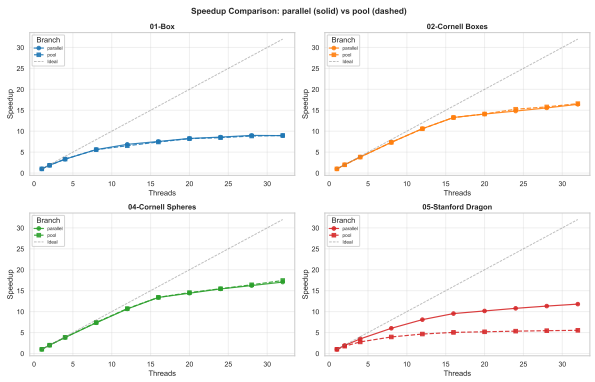

Figure 25: Speedup Total Time - Parallel vs Pool

For the speedup Figure 25, the greatest improvement in efficiency is for scene 05, were the efficiency of using more threads has increased substantially.

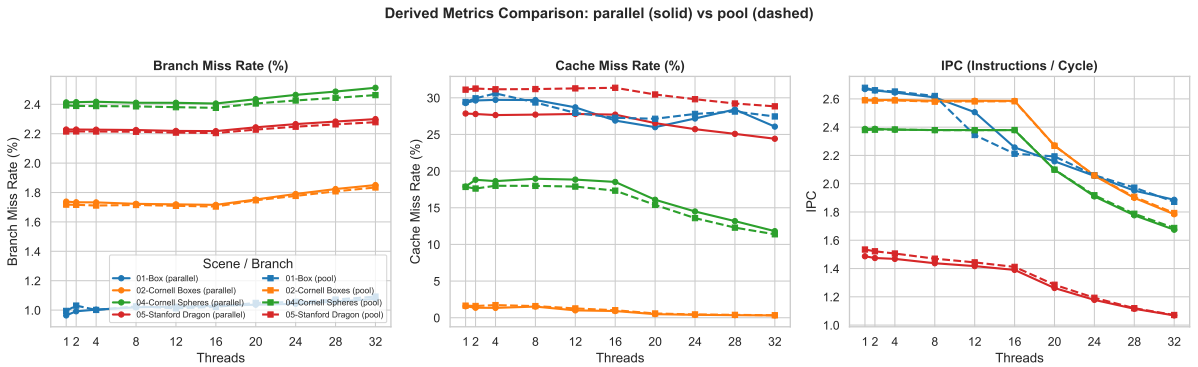

Figure 26: IPC, Branch, Cache Miss Rate Percentage - Parallel vs Pool

There are no things to note for the IPC, Branch, Cache Miss Rate, with the addition of the new improvement. Above behavior is what could be expected when looking at the implementation, and is also what the problem identified earlier tried to solve. Reducing the build phase of the `bvh` tree could be considered a success when looking at the charts in Figure 26.

7. Improvement 5: Structure of Arrays & SIMD

This section will discuss the application of transforming data layout from a AoS to Structure of Arrays (SoA). This to facilitate the application of SIMD vectorization on select functions.

7.1. Problem

As shown in class, using linked lists, which are based on pointers, can result in a problem called pointer chasing. Reducing memory efficiency, when in said lists, elements are regularly removed & added, resulting in pointers being spread throughout several different cache lines. In addition, transforming the data layout allows for the restructuring from AoS to a SoA layout, enabling the application of SIMD.

7.2. Solution

The application of SIMD requires the alignment of allocated memory on either `16`, `32` or `64` bytes depending on available vector instruction support. Implementing the SoA memory layout in combination with `aligned_malloc` allows for the application of SIMD. Furthermore, it would seem attractive to just apply SIMD everywhere, but this is not what the performance numbers indicate. Choosing which function to use SIMD, must be considered & measured.

Due note, that transforming from a AoS to SoA and (naively) replacing all `LINKED_LIST_FOREACH` macro with `for` loops, considerably reduces the performance of the application. Specifically for scene 05 on threads the times were: ∼25s build; ∼60s rendering (1 on Figure 27).

After improving the `aabb_ray_intersect` with a more performant implementation and addition of an `inverse` field on `vec3`, the render time was reduced to ∼50s; the build time did not change (2 on Figure 27).

Let's start of with some examples of functions where the application of SIMD has negligible or negative impacts. The following numbers and examples are measured for `scene-05` and using **20** threads. We note that his is not a statistical analyis, but the wide chance in numbers does give an indication of performance.

Implementing a SIMD based version of `aabb_for_triangles` (including `aabb_for_triangle` & `aabb_surrounding`) the build time regressed to ∼30s. The same can be said for `postprocess_pixels`. There, the SIMD implementation regressed by about ∼1s for the render time (3 on Figure 27).

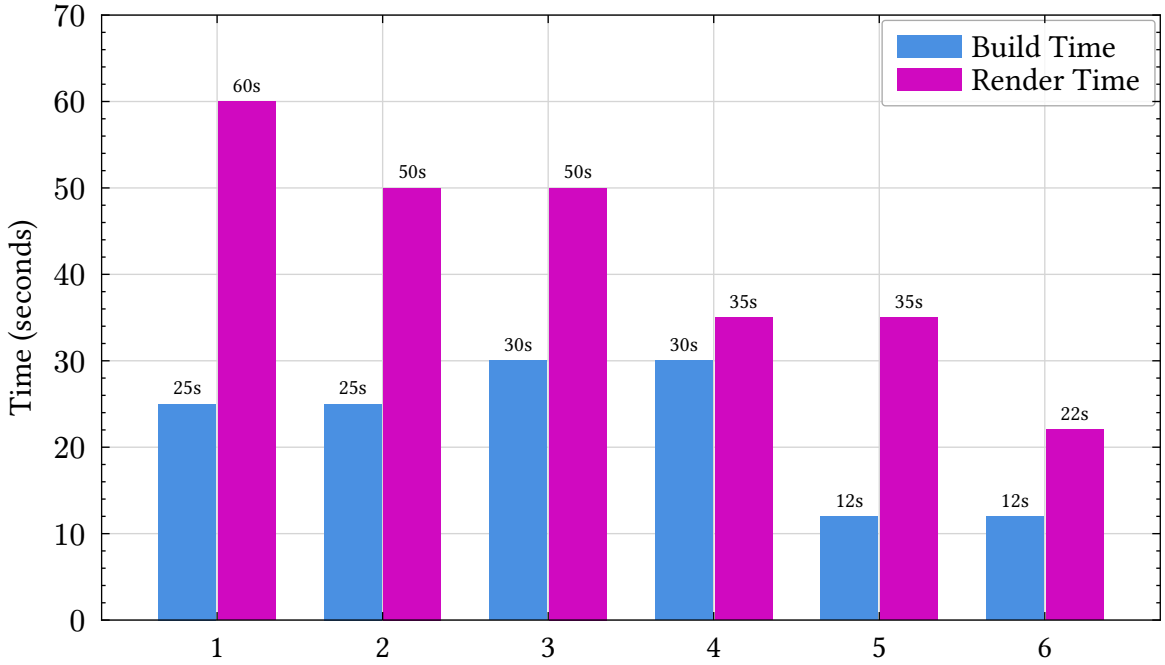

Figure 27: Performance Evolution Profile (Scene 05, 20 Threads)

The functions that did benefit from SIMD application are `linked_list_ray_intersect` (now called `ray_intersect`) and the 2 downstream functions: `triangle_ray_intersect_simd` & `triangle_ray_intersect_sse`. This resulted in the following improvement: render time to ∼35s for scene 05; ∼6s for scene 04, coming from 10/8s (4 on Figure 27).

Continuing with the improvements, applying SIMD on the `calculate_split_cost` function decreased the build time from around ∼25s to ∼12s (5 on Figure 27).

Included in this improvement is the addition of the `inverse` field on the `vec3` struct and re-reordering rays in `bvh_ray_intersect` by returning distance from `aab_ray_intersect`. For scene 05, this resulted in the following improvement; From ∼35s render time to ∼22s (6 on Figure 27).

7.3. Results

The overall execution time with the addition of the SoA & application of SIMD to selective functions has marginally reduced the execution time overall, as seen in Figure 28. Since the improvement does not partially pertain to more efficiency use of threads, the speedup factors between the version in Figure 29, could be considered identical.

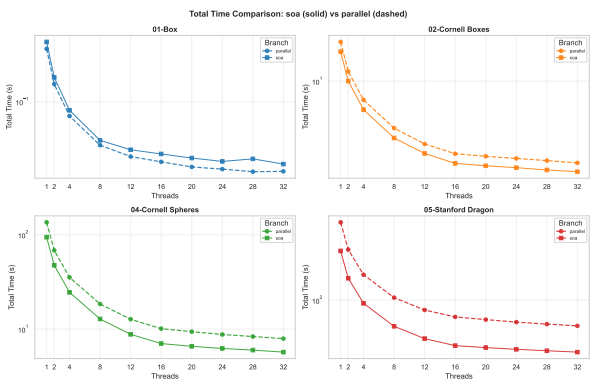

Figure 28: Total Time - SoA vs Parallel

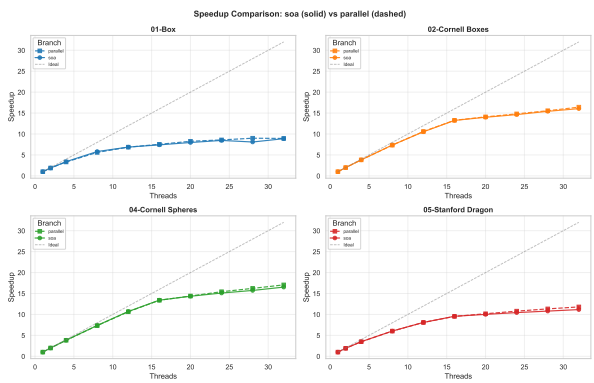

Figure 29: Speedup Total Time - SoA vs Inline

The derived metrics from the `perf stat` wrapper are once again shown in Figure 30.

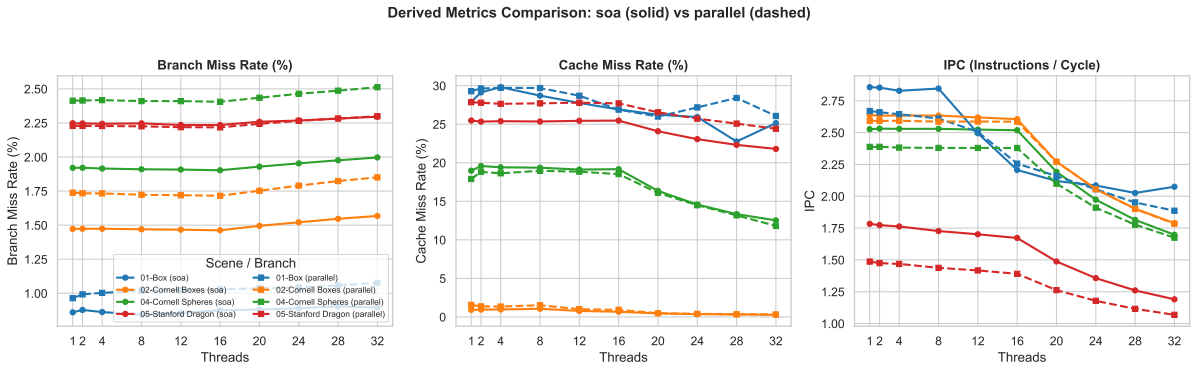

Figure 30: IPC, Branch, Cache Miss Rate Percentage - SoA vs Parallel

The most notable change on the charts to note, is the increase in the IPC for scene-05, with the single thread IPC going from $\sim 1,50$ to ~ 1.75

Across all the scenes and for all metrics, starting from thread count 16, there is an increase in the branch mis prediction rate, decrease in cache-miss rate, but a decrease in IPC.

8. Final

This section will discuss the final version of the pathtracer.

8.1. Improvement

The last improvements for the pathtracer is the addition of an indices based bvh build step. This step is more focused on the reduction of the memory usage, particularly for scene 05. The original implementation in Section 7, the maximum memory usage sits around ∼12GB. The added benefit is a slight improvement in performance during the render time of the image, while build time is the same.

Instead of allocating memory for each leaf of a node during the bvh build step, indices are sorted and passed to their respective child nodes while the main triangles list is used through the build process. When a leaf node is reached, the list of indices is used to load and allocated memory into the leaf nodes triangle list.

8.2. Results

The application execution time can again be seen in Figure 31. The smaller scenes see no real improvement, excluding scene 02 which sees an increase in execution time. In contrast to the largest scene, which has a noticable reduction in execution time.

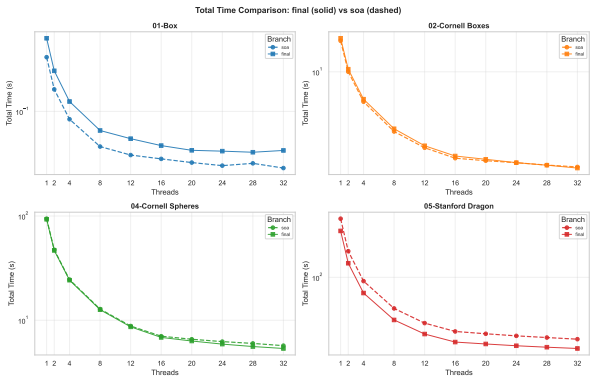

Figure 31: Total Time - Final vs SoA

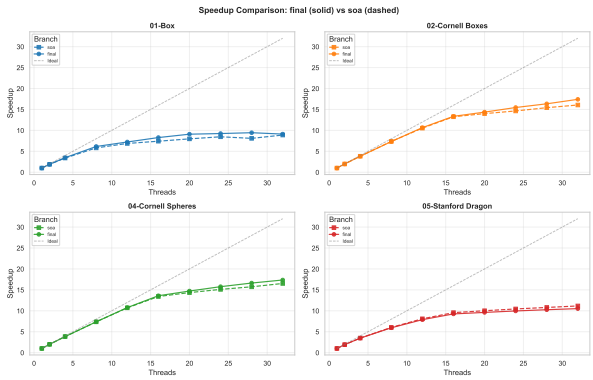

Figure 32: Speedup Total Time - Final vs SoA

There are again no real changes in the speedup charts with the new improvement, as shown in Figure 32.

The derived metrics are visible in Figure 33. The most positive thing to note is that for the larger scenes (04,05), the cache miss rate Percentage has seen a large decrease. For scene 04; going from around 20% to < 10% and for scene 05; going from around 25% to < 20%.

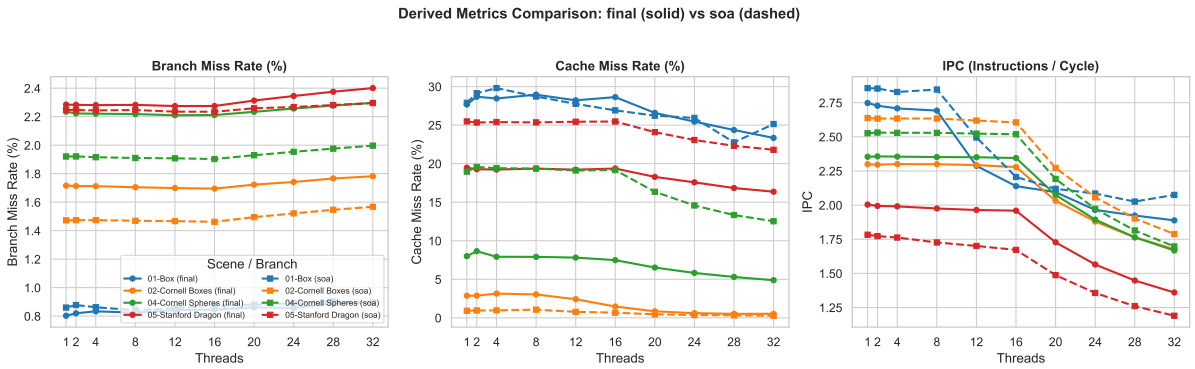

Figure 33: IPC, Branch, Cache Miss Rate Percentage - Final vs SoA

The improvement has a negative impact on the IPC for the scene 01,02,04, and a positive impact for the larger scene 05. All scenes do see an increase in the branch miss prediction rate compared to the SoA version.

9. Conclusion

This section will analyze the different improvements made for the pathtracer application and the overall impact on application performance. The first look will be at the change in total time each scene & stage take in Figure 34.

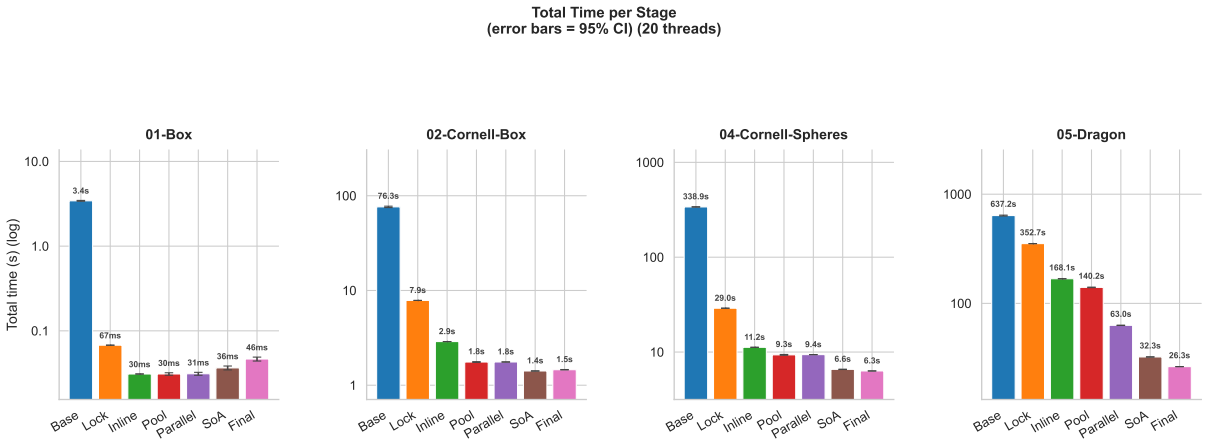

Figure 34:

What is clear for the smallest scene 01, is that from the pool improvement on, each following improvement seems to have a negative impact on the time. This might be attributed to the overhead for the new features dominating.

The additive nature of the improvements appear to have had a positive impact on the execution time of the scenes 02,04 and 05. Ranking said improvements based on impact – shows that the removal of lock contention in `lock` appears to have had the largest impact – with the inlining of `vec3` instructions in `inline` second.

Charting the speedup of each stage relative to the base version, for 3 different thread count (1,20,32), can be seen in Figure 35. The analysis makes use of the geometric mean as seen in class [11].

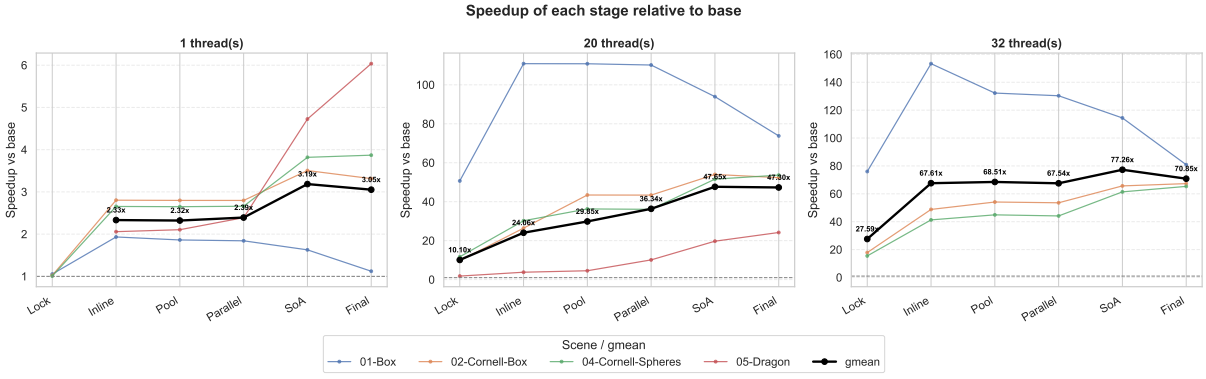

Figure 35: Geometric Mean Speedup, Per Stage & Scene (1, 20, 32 threads)

Due to choices made in data collection, mentioned in Section 2, the 32 thread count chart is incomplete, any conclusions should be guarded. The charts for the 1 & 20 thread count chart, allow for a clear differentiating in speedup between single thread & multi-threaded behavior of the application.

Analyzing the multi-threaded behavior of the final application, using Amdahl's law can be seen in Figure 36. While it is more of a diagnostic tool, the application does allow the analysis of the efficiency of the application for each improvement & scene.

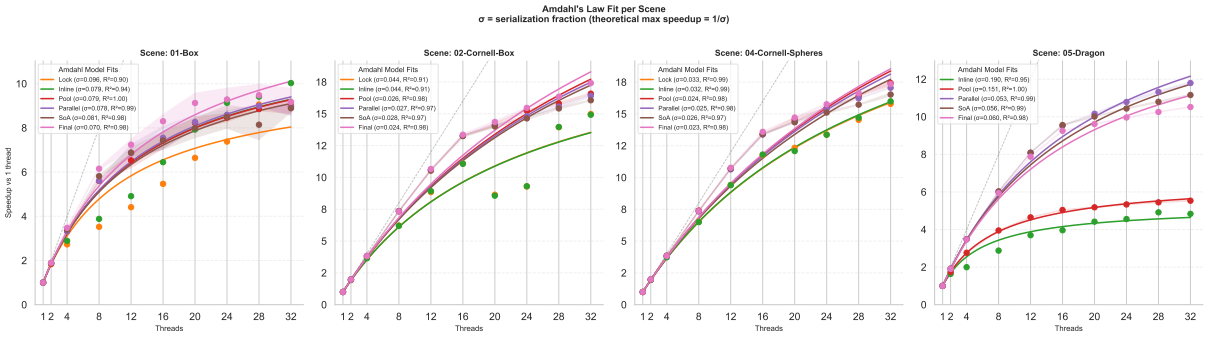

Figure 36: Amdahl Law: Scene x Threads

For the smaller scenes, the behavior does not deviate between the versions. The greatest improvement of efficiency is visible for scene 05, when going from the `pool` version to `parallel`. For each subsequent version (`soa` , `final`) there is a reduction of maximum speedup.

Applying the USL law on the application indicated that Amdahl law provided a better fit for modeling the application's behavior. No performance degradation was observed when increasing the thread count, potentially suggesting that the application could further benefit from a continued increase in thread count for the largest scene.

10. Appendix

10.1. Platform

The benchmarks were executed on a KUbuntu 25.04 desktop, with the specifications listed in Table 1.

Part	Value
CPU	Ryzen 9 5950X
RAM	64GB (3200 Mhz)
OS	Ubuntu 25.10
Kernel Version	6.17.0-23-generic
Scheduling	Default
Make	GNU Make 4.4.1
GCC	gcc (Ubuntu 15.2.0-4ubuntu4) 15.2.0

Table 1: Desktop Specifications

10.2. Charts

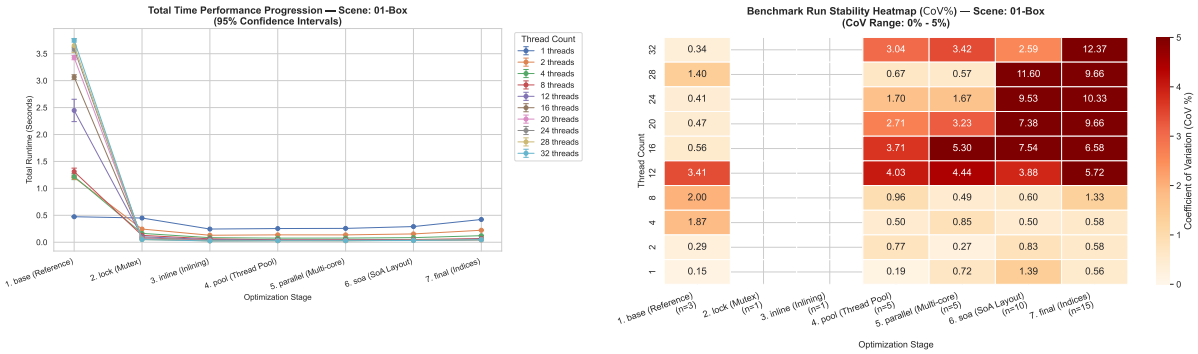

Figure 37: Scene 01 - CI & CoV

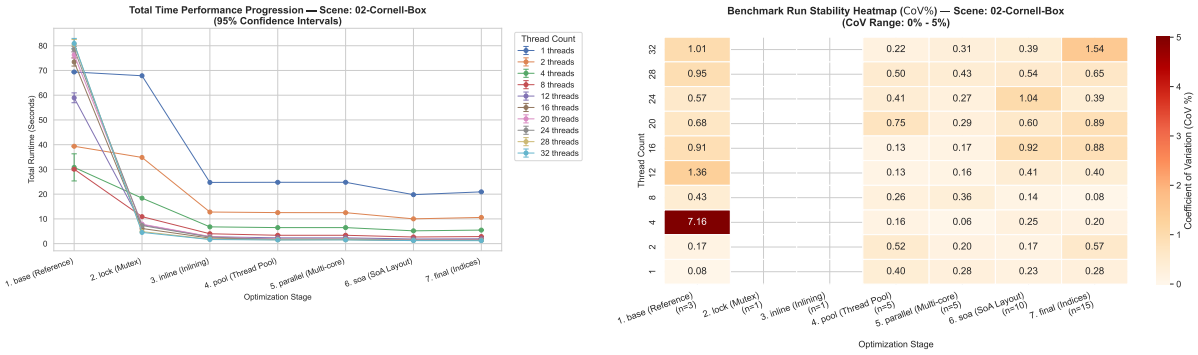

Figure 38: Scene 02 - CI & CoV

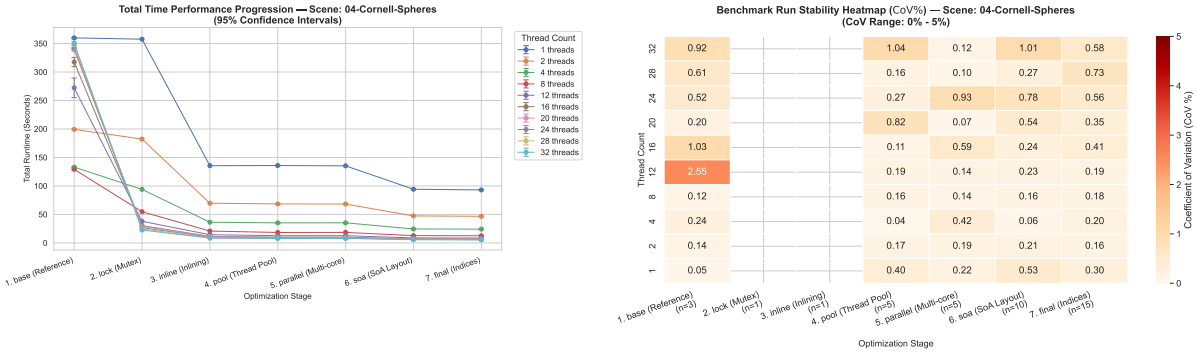

Figure 39: Scene 04 - CI & CoV

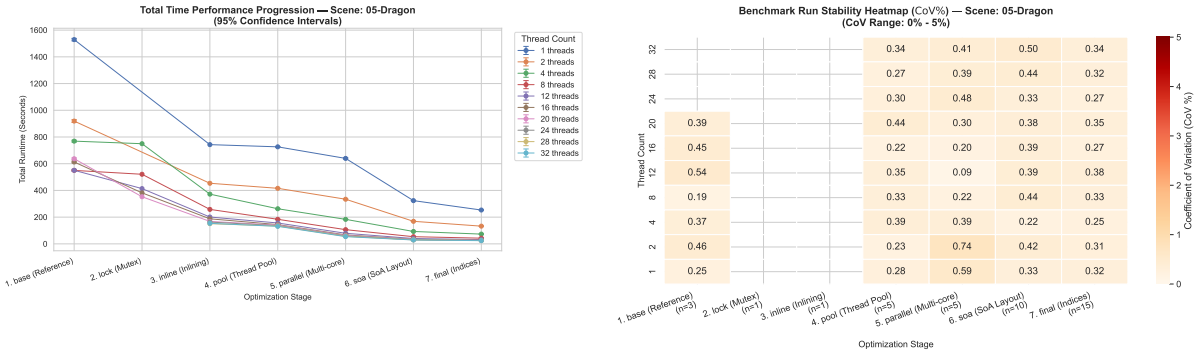

Figure 40: Scene 05 - CI & CoV

Bibliography

[1] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - Coefficient of Variation (CoV).” p. 25, 2025.

[2] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - How Many runs do I need?.” p. 38, 2025.

[3] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - Benchmarking Crimes: A Catalog.” p. 49, 2025.

[4] “rand(3) — Linux manual page.” [Online]. Available: https://www.man7.org/linux/man-pages/man3/rand.3.html

[5] D. Blackman and S. Vigna, “xorshiro128++.” [Online]. Available: https://prng.di.unimi.it/xoroshiro128plusplus.c

[6] “Pseudo-random numbers/Splitmix64.” [Online]. Available: https://rosettacode.org/wiki/Pseudo-random_numbers/Splitmix64

[7] “Inlining.” [Online]. Available: https://en.wikipedia.org/wiki/Inline_expansion

[8] AMD, “AMD µProf.” [Online]. Available: https://www.amd.com/en/developer/uprof.html

[9] John, “Thread Pool in C.” [Online]. Available: https://nachtimwald.com/2019/04/12/thread-pool-in-c/

[10] Unknown, “Creating a Thread Pool in C.” [Online]. Available: https://markc.su/posts/threadpool_c

[11] P. A. Paolillo, “Lecture 2 - Metrics, Distributions & Experimental Rigor - Which Mean When? — Summary.” p. 19, 2025.